

mold: A Massively Parallel Linker

Rui Ueyama

The University of Tokyo

Tokyo, Japan

ruiu@is.s.u-tokyo.ac.jp

Abstract

Linking is a critical step in the software build process that combines compiled object files into a single executable or shared library. Despite decades of engineering effort, link times remain a significant bottleneck in the edit-compile-debug cycle, particularly for large C++ programs. Existing linkers exploit limited parallelism, leaving most CPU cores idle during linking. We present mold, a Unix/Linux linker that applies data parallelism systematically across the entire linking pipeline. We first analyze the architectural constraints that prevent existing linkers from scaling, including entangled symbol resolution and archive processing, and then show how a clean-slate design that decouples them overcomes these limitations. On large real-world programs, mold links multi-gigabyte debug binaries in at most a few seconds, and often in under a second. It is 2.4–16.1x faster than the state-of-the-art lld linker, and up to 112x faster than the traditional GNU ld. An ablation study shows that no single optimization dominates; the speedup comes from the cumulative effect of parallelizing all passes.

CCS Concepts: • **Software and its engineering** → **Compilers**; • **Computing methodologies** → *Parallel algorithms*.

Keywords: linkers, ELF, data parallelism, build systems

1 Introduction

Building large software systems from source code is a multi-stage process that typically involves compiling individual source files into object files and then linking them together into an executable or shared library. Despite extensive research on compilation, the linking step has remained largely neglected. Yet for large programs, linking alone can take tens of seconds, becoming the bottleneck in the edit-compile-debug cycle that developers repeat dozens of times a day. A debug build of TensorFlow, for example, comprises 24 GiB of object files, which the linker combines into a single 9.9 GiB shared library. Linking such a program with the state-of-the-art LLVM lld [35] takes 52 seconds on a 64-core machine, leaving most of its cores idle.

This paper presents mold, a production-quality Unix/Linux linker designed from scratch around a single principle: structure every major pass as a data-parallel loop over homogeneous arrays of elements (symbols, relocations, sections), using concurrent data structures and atomic operations for the few points requiring synchronization. The key architectural decision is to decouple parsing from symbol resolution:

mold eagerly parses all input files—including every member of every archive—in parallel, and only then performs symbol resolution as a separate parallel pass using atomic compare-and-swap. This eliminates the entangled processing that forces sequential execution in prior linkers. The same data-parallel structure is applied to every subsequent pass: relocation scanning, section garbage collection, identical code folding, string merging, layout computation, and output generation. By Amdahl’s law [1], parallelizing only a subset of passes leaves the serial remainder as a hard ceiling on speedup; mold avoids this by parallelizing all of them.

The contributions of this paper are:

- We analyze the architectural constraints that prevent existing linkers from exploiting modern multi-core hardware (§2.3).
- We present a linker architecture that decouples traditionally entangled phases and applies data parallelism to every major pass, and describe the concurrent algorithms used for each (§3, §4).
- We provide a comprehensive evaluation on nine large real-world workloads, including an ablation study quantifying each technique’s contribution, a scalability analysis from 1 to 64 threads, and a head-to-head per-pass comparison with lld (§7).

2 Background

This section outlines the standard linking pipeline, reviews existing linkers and why they are not more parallel, and then introduces key ELF concepts referenced by later sections.

2.1 The Linking Pipeline

A linker reads object files and shared libraries, resolves symbol references, applies relocations, and produces an executable or shared library. The major phases are:

1. **Input parsing:** Read and parse all input object files, shared libraries, and archives. For each file, extract the symbol table and section table.
2. **Symbol resolution:** Build a global symbol table that maps each symbol name to its defining file. Deduplicate COMDAT groups, keeping one copy of each. Resolve references from archives on demand.
3. **Section processing:** Optionally perform garbage collection and identical code folding. Merge string sections, deduplicating identical string literals across input files.

Table 1. Parallelization of major linker passes in the evaluated versions (GNU ld/gold 2.46.1, lld 22.1.8, and mold 2.42.0). ● = parallelized, ◐ = partially or conditionally parallelized, ○ = sequential, — = unsupported.

Pass	GNU ld	gold	lld	mold
Input file parsing	○	●	◐	●
Symbol resolution	○	○	○	●
Relocation scanning	○	○	○	●
Section GC	○	○	○	●
Range extension thunks	○	○	○	●
Identical code folding	—	○	●	●
String merging	○	○	●	●
Section copy + reloc. apply	○	●	●	●
Build-ID computation	○	●	●	●

4. **Layout computation:** Construct synthetic sections (GOT, PLT, dynamic symbol tables), determine the output file layout by computing sizes and offsets of all output sections, and assign virtual addresses.
5. **Output generation:** Copy section contents from input files to the output file, applying relocations to fill in symbol addresses. Write synthetic section contents and file headers.

2.2 Existing Linkers

GNU ld (also known as BFD ld) is the traditional Unix linker, distributed as part of the GNU Binutils package. It is single-threaded and processes input files sequentially, resulting in poor performance on modern hardware.

GNU gold [33] was written from scratch at Google in 2008 as a faster alternative to GNU ld. As an ELF-only linker designed with speed in mind, it achieved significant speedups over GNU ld. gold also introduced multi-threaded support, though it parallelizes only a limited number of passes. After Google migrated to lld, gold lost its primary maintainer and was marked deprecated in 2025 [13].

LLVM lld [35] is the linker from the LLVM project. It parallelizes several passes including some post-parse input-file work, relocation scanning, and output writing. It achieves 2–9x speedup over gold on large programs. However, key passes such as symbol resolution remain sequential [30].

A summary of which passes are parallelized in each linker is shown in Table 1.

2.3 Why Existing Linkers Are Not More Parallel

Given that data parallelism is conceptually straightforward, a natural question is why existing linkers do not already exploit it fully. We identify three reasons.

Order-sensitive semantics. Traditional Unix linking is defined in terms of a single left-to-right pass over the command line. An archive satisfies only the references that are still undefined when the scan reaches it, and an extracted

member may itself introduce new undefined references, so member extraction follows use-def chains across archives that a sequential scan naturally resolves. A single scan cannot handle circularly-dependent archives, so users must wrap them in `-start-group` and `-end-group`, which request that the group be rescanned until no new undefined references are introduced. Reproducing the exact set of archive members extracted by this process is hard with a parallel algorithm in some corner cases.

No formal specification. Unlike programming languages, whose semantics are typically defined by a formal standard, the ELF specification [34] defines only the object file format, not how a linker should process its inputs. Kell et al. [18] give a formal semantics for ELF linking, but their model covers only static linking of small C programs and aims to formalize existing practice rather than define a normative standard. In practice, GNU ld is widely considered the de facto reference implementation.

This absence of a specification makes established conventions hard to change, because it is not clear what is allowed to change. Parallelizing the linker fully can alter corner-case outcomes, such as which definition prevails when two archives wrapped with `-start-group` and `-end-group` define the same symbol. In practice, the only way to evaluate such a deviation is to deploy the linker and see whether anything breaks. Users treat any observable difference as a linker bug, so for an established linker, the experiment is rarely worth the risk. A new linker’s early adopters knowingly accept such differences, which is one reason why drastic improvements tend to come from new linkers rather than existing ones.

Pervasive parallelism is hard to retrofit. gold and lld illustrate the limit of incremental parallelization. Both aim for speed and have received years of optimization. They use threads in passes that can be isolated, yet symbol resolution and other central passes remain sequential, as Table 1 shows. Parallelizing those passes changes the order in which global decisions are made, the representations that carry their results, and the interfaces to every later pass. Making these changes amounts to rewriting much of the linker all at once. At that scale, writing a new linker from scratch can be easier than completing the retrofit.

2.4 Key Concepts

Before describing our design, we briefly review the ELF and linking concepts that the remaining sections build on.

Sections. An ELF object file is divided into named regions called *sections*, containing code (`.text`), initialized data (`.data`), uninitialized data (`.bss`), read-only data (`.rodata`), and various metadata.

Symbols. Symbols are named entities (functions, global variables) that may be *defined* (providing the content) or *undefined* (referencing a definition in another object file).

Relocations. Relocations are metadata instructing the linker to patch specific locations in section data, typically to fill in addresses of symbols whose final locations are not known at compile time.

Archives. An *archive* (or static library, with the `.a` file extension) is a bundle of object files. Unlike object files given directly on the command line, object files in an archive are included in the output only if they define a symbol referenced by another included file. Archives allow programs to link against a large library (such as `libc.a`) without pulling in every function it contains.

Position-independent code, GOT, and PLT. Each process has its own virtual address space, so an executable can be linked to a fixed address and use absolute addresses in its code. Such code is called *position-dependent*. Shared libraries, however, cannot use fixed addresses because multiple libraries loaded into the same process might collide. They are instead compiled as *position-independent code* (PIC) and loaded at an arbitrary base address chosen at runtime. A PIC binary is always loaded as a whole, so relative addresses within it are invariant and can be embedded directly in the binary. However, absolute addresses and references to symbols in other binaries are not known until load time. Today, even executables are often built as position-independent (PIE) to support address space layout randomization (ASLR).

Generating position-independent code requires compiler cooperation because the compiler must avoid emitting code that depends on absolute addresses. When invoked with `-fPIC` or `-fPIE`, the compiler generates position-independent code that uses indirection for references whose targets may not be known at link time. Accesses to such global variables go through the *Global Offset Table* (GOT), a per-binary table of pointers filled in at runtime by the dynamic linker. Calls to such functions go through the *Procedure Linkage Table* (PLT), a set of stubs that jump to addresses resolved at runtime. Both tables are accessed via PC-relative addressing, so they work regardless of where the binary is loaded. The static linker is responsible for scanning all relocations to determine which symbols need GOT or PLT entries and generating these tables accordingly.

COMDAT groups. C++ compilers often emit multiple copies of the same function or data. For instance, when a header file defining a template function is included by multiple source files, each resulting object file contains its own copy of every instantiation of that template. These duplicate definitions are placed in *COMDAT groups*: sets of sections tagged with a group name so that the linker can keep one copy and discard the rest. In large C++ programs, COMDAT

deduplication is a significant part of the linker’s work; a Firefox debug build contains 2.7 million COMDAT groups across its input files, of which 1.6 million are redundant copies that the linker discards.

Fine-grained sections. When compilers are invoked with `-ffunction-sections` and `-fdata-sections`, each function and global variable is placed into its own section rather than being grouped into a single large `.text` or `.data` section. This enables fine-grained garbage collection (discarding individual unreferenced functions) and identical code folding (merging functions with identical machine code) by the linker.

Range extension thunks. On typical RISC architectures, instructions are at most 4 bytes wide, which leaves a call instruction only a modest displacement field to encode the distance to its target. ARM64’s BL (branch-with-link) instruction, for example, has a 26-bit offset, so a direct call can reach only targets within ± 128 MiB of the call instruction. Whether a call’s target lies within that range is not known until the linker lays out the output file.

The linker therefore checks each call relocation during layout. When a target is out of reach, it inserts a *range extension thunk* (also called a *veneer* or *trampoline*) [3, 31] near the call site and redirects the original call to the thunk. Each thunk is a short code sequence that loads the full target address into a register and performs an indirect jump, as in this ARM64 example:

```
caller: bl    thunk                // was 'bl callee'
...
thunk:  adrp  x16, callee
        add   x16, x16, :lo12:callee
        br   x16                  // indirect jump
```

Among mold’s supported targets, ARM 32/64 and PowerPC 32/64 require range extension thunks.

3 Design Principles

A linker’s workload is dominated by operations on large, homogeneous arrays of data: tens of millions of relocations, millions of symbols, hundreds of thousands of sections. Each element can typically be processed independently or with only lightweight synchronization. This structure is ideally suited to *data parallelism*: applying the same operation to each element of a large array in parallel.

The design of mold is built around this data parallelism. Rather than trying to overlap different phases (task parallelism), mold executes passes *serially* but parallelizes *within* each pass using parallel-for loops. This serial-pass structure is easy to reason about: at any point in execution, exactly one pass is running, and its invariants are easy to state. gold took the opposite approach, employing task parallelism through a work queue with dependency tokens [33]. In principle,

Table 2. Symbol precedence rules for parallel symbol resolution, from highest to lowest. Ties within the same rank are broken in favor of the definition whose input file appears earlier on the command line.

Rank	Symbol type
1	Strong defined symbol
2	Weak defined symbol
3	Strong defined symbol in a shared library or archive
4	Weak defined symbol in a shared library or archive
5	Common symbol
6	Common symbol in an archive
7	Undefined (no definition seen)

independent tasks (e.g., reading different input files) can run concurrently. In practice, however, most tasks are serialized by their dependency chains, and benchmarks show negligible speedup or even minor slowdowns from multi-threading. gold disables multi-threading by default for this reason. The lesson is that the real opportunity lies not in overlapping coarse-grained phases but in processing millions of homogeneous elements within a single phase.

4 Parallelizing the Linking Pipeline

We now describe how each major linker pass is parallelized in mold.

4.1 Input File Parsing

To allow symbol resolution to run as a single parallel pass (§4.2), mold fully parses all input files, including archive members, upfront and in parallel.

Parsing reads the ELF header, section headers, and symbol tables; relocation tables are located but not processed until later passes. Symbol names are *interned* so that files that define or reference the same name share one symbol object, reducing symbol identity to pointer equality. These pointers form a parallel array alongside each file’s ELF symbol table, and all later passes iterate over the interned symbols rather than the raw ELF entries. Interning itself is parallel: names are binned by hash into thread-local buffers during parsing, and each bin is deduplicated independently afterwards, with no shared concurrent data structure.

4.2 Symbol Resolution

Symbol resolution determines, for each symbol, which definition prevails. Each file tries to install itself as the *owner* of the symbols it defines, using an atomic compare-and-swap on the owner field of the shared symbol object. When multiple files define the same symbol, the precedence rules in Table 2 decide the winner.

The ELF specification defines basic precedence rules (e.g., strong definitions override weak ones) but leaves many interactions among regular object files, archive members, and

shared libraries unspecified. The rules in Table 2 are therefore heuristic. We tried several orderings and kept the one that disagrees least with traditional linkers in practice. We validated this choice by building the entire Gentoo repository, where only two of more than 19,000 packages failed to build because of this difference in resolution semantics (§8).

After symbol resolution, archive member inclusion is determined by a liveness walk. Because the preceding compare-and-swap phase has already set each symbol’s owner to the file with the strongest definition, every defined symbol now holds a direct pointer to its owning file. The walk is therefore straightforward: non-archive input files are unconditionally marked live, and when a file becomes live, the linker follows the owner pointers of the symbols it references to mark the defining files live in turn. This makes the linker largely insensitive to the order of `-l` flags: unlike a left-to-right scan of the command line, the walk can reach an archive member that appears earlier on the command line than the file that references it, and it follows circular dependences among archives without any rescanning. mold accepts `-start-group` and `-end-group` for compatibility, but they have no effect.

COMDAT deduplication works similarly. Group signatures are interned like symbol names, so files that define the same group share one group object. Live files then claim each group with an atomic compare-and-swap, and files that lose discard their copies of the group’s member sections.

4.3 String Merging

ELF object files may contain *mergeable string sections*, which hold string constants that the linker should deduplicate across object files. In a Firefox debug build, there are roughly 21 million such strings, of which nearly three quarters are duplicates; merging them can account for a significant fraction of the end-to-end link time.

mold deduplicates strings by inserting them into a concurrent hash map in parallel across all input files. We found that the general-purpose concurrent hash map in oneAPI Threading Building Blocks (oneTBB) [16] was not fast enough for this workload, so we wrote a specialized one ourselves. The main optimization is to avoid hash table resizing: we first estimate the number of distinct strings using the HyperLogLog algorithm [10] and allocate a table large enough from the start. All input files then insert their strings in parallel using atomic compare-and-swap; identical strings are mapped to the same entry.

4.4 Relocation Scanning

The linker must scan relocations to determine which symbols require entries in the GOT, PLT, or other synthetic sections. Whether an entry is required depends on the outcome of symbol resolution, not on the relocation type alone. For example, a relocation of type `R_X86_64_PLT32` permits a PLT entry rather than demands one: if the symbol was resolved to a definition in the output executable itself, the call is bound

directly and no PLT entry is needed. For the Firefox debug build, this pass examines 11 million relocations.

Relocation scanning in mold is a parallel-for loop over the input sections. For each section, each relocation is examined, and if the referenced symbol requires a GOT or PLT entry, an atomic flag on the symbol object is set. Since each symbol's flag is set monotonically (from “no entry needed” to “entry needed”), relaxed atomic bitwise-OR operations suffice; no locks are required.

After the parallel scan, the symbols with set flags are collected to construct the GOT and PLT. This construction step is fast because the number of symbols requiring GOT/PLT entry is typically orders of magnitude smaller than the total number of symbols.

4.5 Section Garbage Collection

When programs are compiled with `-ffunction-sections` and `-fdata-sections`, the linker can perform garbage collection (GC) to discard unreferenced sections. This is modeled as a mark-sweep traversal of a directed graph where sections are vertices and relocations are edges.

mold parallelizes the mark phase using a parallel-for-each loop with a feeder pattern. The algorithm starts from a set of root sections (those reachable from the entry point, exported symbols, and sections with special semantics like `.init_array`) and performs a parallel reachability traversal. When a thread marks a section as reachable, it feeds the section's relocation targets into the work queue so they are visited in turn.

4.6 Identical Code Folding

Identical Code Folding (ICF) is an optional size optimization that merges read-only sections with identical contents and relocations. The challenge is that section identity is *recursive*: for two sections to be identical, their relocation targets must themselves be identical. This is equivalent to computing bisimulation equivalence over a directed graph in which sections are vertices, relocations are edges, section contents are vertex colors, and relocation types are edge labels.

mold computes this equivalence with the hash-based refinement algorithm that Schätzle et al. [28] used for large-scale bisimulation reduction with MapReduce. The algorithm is a form of *color refinement*, also known as the one-dimensional Weisfeiler-Leman algorithm [36]. It works as follows. On iteration N , each section's cryptographic hash summarizes all walks of length up to N originating from it. The first iteration hashes the section's contents, flags, and relocation types, but not their targets. Each subsequent iteration computes a new hash from the section's previous hash and the previous hashes of its relocation targets, extending the walk length by one. The hash sequence is monotonically refining: each iteration can only distinguish sections that were considered equal in the previous iteration, never merge previously distinct ones. The process therefore converges

when the number of distinct hashes stops increasing, and sections with the same hash at that point are considered identical. Cryptographic hashing makes erroneous merges due to hash collisions negligibly unlikely.

Unlike classical partition refinement algorithms such as Hopcroft's [15], which explicitly split equivalence classes, hashing avoids pairwise comparison, and every section computes its new hash from the previous iteration's values alone. Each iteration is embarrassingly parallel and needs no synchronization.

4.7 File Layout Computation

Before copying input section contents to the output file, the linker must assign an output file offset to every input section. This is essentially a prefix sum over section sizes, but with two complications: input sections have alignment requirements, and a single output section may contain millions of input sections, making a sequential scan a bottleneck.

mold uses the standard two-level parallel scan [4]. First, input sections within each output section are split into groups of roughly 10,000. The total size and alignment of each group are computed in parallel. Second, group offsets within the output section are assigned by a serial scan over these groups, which are far fewer than the individual sections. Third, input section offsets within each group are assigned in another parallel pass, now that each group's base offset is known. Finally, file offsets for the output sections themselves are assigned serially, which is trivial because a typical link produces only a few dozen output sections.

Executable sections on architectures that need range extension thunks are laid out differently, as described next.

4.8 Range Extension Thunks

On some architectures, the linker must insert range extension thunks (§2.4) into the code segment when a call's displacement exceeds the instruction's immediate field. The challenge is that inserting a thunk makes the code larger, which may push previously in-range calls out of range, requiring yet more thunks.

GNU ld, gold, and lld all create thunks with essentially the same sequential fixed-point algorithm: lay out the code first, then insert thunks into it. lld, for example, pre-allocates empty thunk groups at regular intervals and scans all relocations in a single thread [29, 31]. When a call turns out to be out of range, lld adds a thunk to the lowest-addressed group reachable from the call. Each insertion, however, lengthens that group and pushes everything after it to higher addresses. A call that was scanned earlier and left as a direct branch may have its caller and callee on opposite sides of that group; its displacement has just grown, and it may no longer be in range. A single scan therefore does not suffice: lld rescans all relocations, inserting more thunks as needed, until a scan finds nothing new, which usually takes a few iterations.

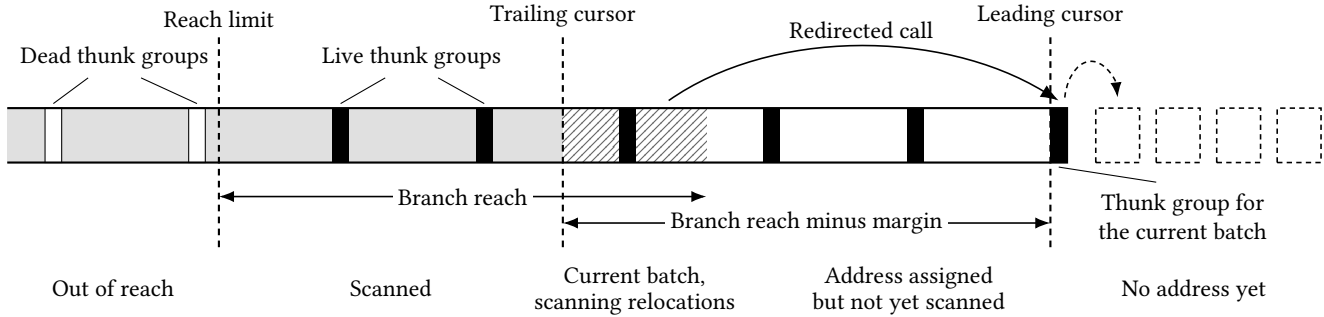


Figure 1. Linear-scan thunk creation in mold. Addresses are assigned up to the leading cursor, and relocations are scanned in batches at the trailing cursor, which stays less than one branch reach behind. Each batch emits one thunk group at the leading cursor. A thunk group stays live until the scan has advanced one branch reach past it.

mold eliminates the repetition by creating thunks in a single linear scan during layout rather than after it. Figure 1 illustrates the algorithm. Within each output section, mold assigns addresses to input sections in increasing order, while maintaining two cursors over the input section list, one leading and one trailing. The leading cursor points to the end of the section that has just received an address, the highest assigned so far. The trailing cursor points to the sections whose relocations are being scanned. The gap between the two cursors is one branch reach (e.g., 128 MiB on ARM64) minus a small margin reserved for the thunks themselves. The scan decides whether each call needs a thunk. A call that is out of range, or whose target has no address yet, is redirected to a live thunk for its target if one exists; otherwise a new thunk is added to the group at the leading cursor and the call is redirected there. Inserting a new thunk cannot invalidate any earlier decision, because every call scanned so far has both its caller and its destination (the callee or an earlier thunk for it) at lower addresses than the new thunk.

mold advances the trailing cursor in batches so that each batch’s relocations can be scanned in parallel. Entries are deduplicated with a per-symbol atomic flag.

During the scan, calls into other output sections are pessimistically considered out of range. This assumption is an optimization: it makes every output section self-contained, so each one’s internal layout can be computed independently of the others. This pessimism can overallocate thunk entries. Once the global layout is known, mold rescans the relocations in parallel and removes a symbol from every thunk group if no call to it actually needs a thunk. Section sizes and offsets are then recomputed. This grow-then-shrink order is safe: removing entries can only leave a displacement unchanged or make it smaller, whereas inserting entries at this stage could make an existing branch overflow.

Even though mold’s algorithm is simpler and computationally cheaper, its output is comparable to that of the fixed-point algorithm: for the ARM64 Firefox debug build, mold

emits 19,080 thunk entries and lld 18,896. The small discrepancy, about 2 KiB in a 282 MiB text segment, is due to minor differences in output layout between the two linkers.

4.9 GDB Index Construction

A `.gdb_index` section indexes symbol names and address ranges in DWARF [7] debug information, allowing GDB to start quickly without scanning all of the DWARF data. When `-gdb-index` is specified, mold constructs the index.

The index was originally generated by GDB’s post-link tool `gdb-add-index` [11], which reads a completed binary and adds the section. Linkers later gained the ability to construct it during linking, but the dependency remained one-way. Index construction consumes what the linker has generated, and no linker pass consumes the index. mold therefore builds it concurrently with other passes, the only major exception to its serial-pass structure (§3).

4.10 Output Generation

In the final phase, mold memory-maps the output file and processes all input sections in parallel: each input section copies its data to the output buffer and applies its relocations in place. This is naturally parallelizable because each input section writes to a disjoint region of the output.

The last piece of the output is the build ID. ELF executables commonly contain a `.note.gnu.build-id` section that uniquely identifies the binary; the linker is free to choose any method to compute the identifier as long as it is unique, and a cryptographic hash of the output file is typically used. mold parallelizes the hash computation using a two-level Merkle tree [23]. It computes a first-level hash over each 4 MiB block of the output file in parallel, and then hashes the concatenation of the block hashes to produce the final identifier. For the hash function, mold uses BLAKE3 [24], a cryptographic hash function designed to take advantage of SIMD instructions on modern processors. Hashing the 2.5 GiB Firefox debug output takes 23 ms on our test machine (§7.1).

4.11 Deterministic Output

A parallel linker can easily produce nondeterministic output. For example, when multiple threads insert strings into a shared string pool concurrently, the order of strings within the pool varies with thread scheduling; a linker that assigns output offsets to the strings in pool order will then emit them in a different order on each run. Each ordering is correct—the linked binary works regardless—but the output is no longer bit-identical across runs, breaking build reproducibility [27].

mold avoids this by designing each parallel phase so that its result does not depend on the order in which threads execute. This is achieved either by using commutative operations (e.g., setting atomic flags on symbols) or by following a nondeterministic parallel step with a pass that sorts its results into a canonical order. As a result, mold guarantees that the same inputs, command-line options, and linker version always produce a bit-identical binary.

5 System-Level Optimizations

Beyond the parallel pipeline described above, mold employs several system-level optimizations to reduce link time.

5.1 Huge Pages for the Output File

mold writes the output file through a memory mapping. On Linux, every page of a fresh output file is created on first touch by a minor page fault, which adds up to more than a million faults for a multi-gigabyte output, and concurrent fault handling contends on per-file kernel locks. mold therefore asks the kernel to back the output mapping with huge pages using `madvise(MADV_HUGEPAGE)`. On kernels that can use huge pages for file-backed memory, as recent Linux kernels can for common filesystems such as ext4, the output is then faulted in huge-page units rather than base pages (e.g., 2 MiB versus 4 KiB on x86-64), cutting the fault count by orders of magnitude. This single `madvise` call makes mold's output copy pass 3.1x faster and shortens the Firefox debug link from 1.46 s to 0.89 s.

The optimization is easy for any linker to adopt, which raises the question of how much of mold's advantage it explains. The answer is governed by Amdahl's law. We applied the same one-line change to lld, and its time for the same link improved by 12% (7.04 s to 6.19 s), versus 39% for mold. Output copy accounts for about a quarter of lld's largely sequential runtime, so even a large improvement to that pass moves the total only so far. A system-level optimization like this pays off in proportion to how little other work remains, and pervasive parallelism leaves mold with far less of it than lld.

5.2 Output File Block Preallocation

A freshly created output file is *sparse*. That is, `ftruncate` sets its size, but no disk space is assigned until the file is actually written. The first write to each page of the mapped output

therefore enters the page-fault handler, which must reserve disk space for the page. The filesystem in our evaluation, ext4, serializes these reservations on a per-file lock, so dozens of threads writing a multi-gigabyte output spend more time spinning on the lock than copying. mold avoids this by pre-allocating the entire file with one `fallocate` system call before mapping it; preallocating extents for the 2.5 GiB Firefox debug output takes about 5 ms. The call makes mold's copy pass 3.5x faster (0.82 s to 0.24 s) and shortens the Firefox debug link from 1.46 s to 0.89 s, coincidentally the same figures as in §5.1. Consistent with the proportionality rule of §5.1, the same one-line change applied to lld shortens the same link by only 3.8% (7.04 s to 6.77 s).

5.3 Memory Allocator

A parallel linker stresses the memory allocator, as dozens of threads allocate and free small objects concurrently. mold uses `mimalloc` [20], an allocator designed for concurrent workloads. The choice is worth a factor of 1.3: linked against the glibc malloc instead, mold takes 1.15 s rather than 0.89 s on the Firefox debug build. Other scalable allocators land in between: `jemalloc` [8] at 1.03 s, `tcmmalloc` [12] at 1.09 s, and `tbbmalloc` [16] at 1.12 s.

To gauge how close `mimalloc` comes to the practical limit for this workload, we also implemented a simple bump-pointer allocator that allocates from large per-thread buffers and never frees individual objects. It was in fact slower (1.00 s) and increased peak memory consumption by 2%. This result suggests that replacing `mimalloc` with another general-purpose allocator is unlikely to yield a substantial further speedup.

Switching from the glibc malloc to `mimalloc` helps lld too, shortening its time on the same workload from 7.04 s to 5.97 s. Memory allocation, unlike output copy, occurs in every pass, so by the proportionality rule of §5.1, lld gains more from the allocator than from either output file optimization.

5.4 Output File Reuse

In an edit-compile-debug cycle, the output file the linker is about to write usually already exists from the previous build. Writing a new file and renaming it over the old one implicitly deletes the old file, and deleting a multi-gigabyte file is not cheap: the kernel must tear down its page cache and release its disk blocks. mold therefore overwrites the existing file in place when it can, reusing its inode, its disk blocks, and its page cache. Linux refuses to open a running executable for writing, so overwriting cannot corrupt a program that is currently running; if the open fails, mold falls back to creating a new file. Shared libraries have no such protection, and a running process may have the old file mapped, so for shared library outputs mold always creates a new file. Firefox's main binary is a shared library, so we measure this optimization on Chromium's 4.5 GiB debug executable instead: overwriting saves about 0.3 s.

5.5 Two-Process Architecture

When a linker calls `_exit`, the kernel reclaims all the resources the process has been holding, tearing down its address space in a single thread, and notifies the parent of the termination only after that. Since a linker is a large process with many memory-mapped files, the reclamation takes an appreciable amount of time. The parent is the build system waiting for the linker to finish, so that time is perceived as link time. mold reduces the teardown cost by dropping its file mappings in parallel with `madvise(MADV_DONTNEED)` before exiting (unlike `munmap`, `madvise` does not serialize on the exclusive address-space lock on Linux); the contents remain in the page cache, so the next link still finds the files warm. The technique is linker-independent: retrofitted into `lld` (§7.1), it shortens the Firefox debug link by 0.3 s.

The remaining teardown latency is hidden by a two-process architecture. At startup, mold forks a child process that performs the actual linking while the original process waits. When the child finishes writing and closing the output file, it signals the waiting process, which exits immediately, returning control to the build system; the parent’s teardown is negligible because it holds almost no resources. The child then exits, and the kernel reclaims its resources in the background, invisible to the user. On top of the exit cost reduction described above, the two-process architecture saves a further 10% of the user-perceived link time, for example 0.10 s for a Firefox debug link.

6 Implementation

mold is implemented in approximately 28,000 lines of C++20 code shared across all targets, plus 300–1300 lines per target architecture. It uses oneTBB for parallel execution.

6.1 Compact Metadata Representation

mold keeps its in-memory linker metadata compact to reduce memory traffic and cache misses. Objects such as input files, symbols, and sections are allocated in a dedicated arena and refer to one another using 32-bit offsets within the arena instead of 64-bit pointers. This halves the storage required for each cross-reference and reduces the size of metadata records traversed by many linker passes.

6.2 Architecture-Parametric Design

mold supports 14 processor architectures (i386, x86-64, ARM 32/64, RISC-V 32/64, PowerPC 32/64, LoongArch 32/64, s390x, SPARC64, m68k, and SH-4) through C++ templates. The core linker logic is parameterized on an architecture type `E` that provides types for ELF data structures (e.g., headers, symbols, relocations) and constants (e.g., relocation type enumerations, page sizes). Architecture-specific code is implemented in separate files (`arch-*.cc`).

This design avoids the overhead of virtual dispatch at the cost of a larger linker binary, since the core logic is instantiated once per target architecture.

6.3 Link-Time Optimization Support

mold supports GCC and LLVM Link-Time Optimization (LTO). At an early stage of the linking pipeline, mold loads the respective compiler’s LTO plugin via `dlopen` and delegates intermediate-representation compilation to it, producing native object files that are then linked normally. When LTO is enabled, the plugin’s compilation step dominates the end-to-end link time, so mold’s speed advantage is diminished. In practice this matters little, because LTO is typically reserved for optimized release builds, not the debug builds where linking speed is most critical.

7 Performance Evaluation

We evaluate mold’s performance along four dimensions: end-to-end performance on real-world workloads (§7.2), scalability and CPU cost (§7.3), an ablation study (§7.4), and a per-pass comparison with `lld` (§7.5–§7.6).

7.1 Experimental Setup

Experiments were conducted on two machines. The primary machine has an AMD Ryzen Threadripper 7980X (64 cores, 128 threads), 384 GiB of DDR5 RAM, running Linux 6.17 on Ubuntu 24.04. The second machine is an Apple Mac Studio with an M1 Ultra SoC (16 performance cores and 4 efficiency cores) and 128 GiB of unified LPDDR5 memory, running Fedora Asahi Remix 42 with Linux 6.18. On both machines, the benchmarks reside on an ext4 filesystem on NVMe storage. Frequency boost is disabled, the frequency governor is pinned to performance, and transparent huge pages are in `madvise` mode. Unless otherwise noted, results are from the x86-64 machine. We compare four linkers:

- mold 2.42.0
- lld 22.1.8, with mold’s system-level optimizations retrofitted (see below)
- gold 2.46.1
- GNU ld 2.46.1

The system-level optimizations of §5 require care in how they enter the comparison. Adopting huge pages (§5.1), file preallocation (§5.2), the memory allocator (§5.3), and the exit-time teardown of file mappings (§5.5) requires no architectural restructuring, so they are available to other linkers as-is. To separate their effect from mold’s architecture, we modify `lld` to use all four and measure it in this retrofitted configuration throughout; the reported gaps then reflect architectural differences. The four optimizations compose roughly additively, making `lld` 1.1–2.1x faster than as shipped across our benchmarks; the Firefox debug link, for example, shortens from 7.04 s to 4.44 s. GNU ld and gold are measured unmodified: they are one to two orders of magnitude behind

Table 3. Benchmark workloads and their input and output sizes. Sizes are in GiB.

Program	Number of object files	Release		Debug	
		In	Out	In	Out
Blender 5.2	9,943	0.90	0.25	8.75	2.52
Chromium 145	40,945	2.20	0.53	12.31	4.53
Clang 21.1	3,030	0.45	0.21	18.11	4.25
ClickHouse 26.1	14,280	3.17	1.21	12.82	5.59
Firefox 149	3,155	0.60	0.22	6.48	2.47
Godot 4.6	2,682	0.46	0.15	4.40	1.11
LibreOffice 26.2	7,185	0.58	0.19	4.07	1.00
PyTorch 2.9	2,793	0.69	0.32	9.87	3.60
TensorFlow 2.21	8,420	1.83	0.81	23.87	9.91

on every workload, and the retrofit would not change that picture.

The other two optimizations of §5 depend on conditions a benchmark must control: output file reuse helps only when a previous output exists on disk, and the two-process architecture does not speed up linking but hides the process teardown latency from the build system. We disable both, by removing the output file before each run and by passing `-no-fork` to mold, so that every linker is measured on the same task: building the output from scratch and terminating.

We selected workloads from nine large, widely used open-source programs. Table 3 lists them along with their input and output sizes. We measure the time to link each program’s main binary in two configurations: a *debug* build with debug information and a *release* build without it. These produce output binaries ranging from 0.15 GiB to 9.91 GiB, making link time a meaningful fraction of the overall build cycle.

Link time matters most for large debug builds, which developers relink repeatedly in the edit-compile-debug cycle. Speeding up such links is mold’s primary design goal. The detailed analyses in §7.3–§7.5 use the Firefox debug build as a running example. We do not evaluate link-time optimization workloads because LTO is typically used only for production builds, where the LTO compilation step itself dominates total time and mold’s advantage is negligible (§6.3).

Each measurement is the median of five runs after one warmup run that brings input files into the page cache, reflecting the realistic scenario in which object files remain cached from the immediately preceding compilation.

7.2 End-to-End Performance

Table 4 shows end-to-end link times for the release and debug configurations of each program, with every linker at its default thread count. Speedups generally grow with link size. Small links are dominated by startup costs that both linkers pay alike, and as links grow, more of lld’s time goes to its single-threaded passes (§7.5). The largest speedup, 16.1x on

Table 4. End-to-end link times in seconds; for each program, the first row is the release build and the second row the debug build. Lower is better. Speedup is mold’s speedup over lld. A “—” indicates that the linker cannot link the workload. Here and in all following tables and figures, lld denotes the retrofitted configuration described in §7.1.

Program	mold	lld	gold	GNU ld	Speedup
Blender	0.22	0.82	5.14	11.32	3.7x
	0.98	4.14	55.17	74.40	4.2x
Chromium	1.05	6.29	—	—	6.0x
	1.89	13.24	—	—	7.0x
Clang	0.13	0.37	3.15	6.66	2.8x
	1.48	4.54	91.31	81.84	3.1x
ClickHouse	0.47	2.66	—	—	5.7x
	1.20	4.87	—	—	4.1x
Firefox	0.25	0.98	—	14.33	3.9x
	0.89	4.44	—	92.86	5.0x
Godot	0.11	0.34	2.49	6.02	3.1x
	0.51	1.20	20.56	26.64	2.4x
LibreOffice	0.26	0.98	3.09	29.13	3.8x
	0.46	2.50	23.20	—	5.4x
PyTorch	0.17	0.47	3.49	7.47	2.8x
	0.88	2.60	47.65	59.97	3.0x
TensorFlow	0.75	10.45	—	—	13.9x
	3.23	52.16	—	—	16.1x

the TensorFlow debug build, combines size with a workload-specific cause: the link applies a version script with two dozen glob patterns, which lld matches against 2.5 million defined symbols one pattern at a time on a single thread. That pass takes about 30 of lld’s 52 seconds. Even without it, the speedup would be about 7x. The second largest, Chromium’s debug build at 7.0x, is a pure case of size: its link has by far the most input files in the suite (40,945; Table 3), and lld spends about 7 of its 13.2 seconds parsing them. The other programs gain 2.4–5.7x.

The empty cells in Table 4 mark configurations that a linker cannot link. The GNU linkers fail on different workloads for different reasons, each of which falls into one of four categories: the workload uses a command-line option implemented only by lld and mold, relies on their order-insensitive archive symbol resolution (§4.2), references a symbol synthesized only by them, or uses a new ELF feature that the GNU linkers do not yet support. That 7 of our 18 workload configurations cannot be linked by GNU ld suggests that, for large modern C++ projects, lld rather than GNU ld has become the practical compatibility baseline.

Table 5 lists peak memory usage on the debug builds. mold, lld, and gold perform I/O through `mmap` and reach similar peaks because the dominant term is one they share: the

Table 5. Peak memory usage (maximum resident set size) in GiB when linking the debug builds.

Program	mold	lld	gold	GNU ld
Blender	15.51	15.62	18.55	9.37
Chromium	23.57	26.44	—	—
Clang	25.62	26.75	32.63	13.61
ClickHouse	21.09	22.35	—	—
Firefox	11.24	12.07	—	10.64
Godot	6.69	6.89	8.48	4.37
LibreOffice	6.57	7.40	6.82	—
PyTorch	15.41	16.43	19.38	9.86
TensorFlow	37.35	40.60	—	—

resident pages of the mapped input and output files. mold’s eager parsing of every archive member adds little, because it reads only the member’s symbol and section tables, not the section contents. GNU ld performs I/O with explicit read and write calls, and its peak RSS on the large debug links is therefore markedly lower.

7.2.1 ARM64 Results. Table 6 shows end-to-end link times on the Apple M1 Ultra, an ARM64 machine with 16 performance cores (restricted to those cores via taskset). The speedup of mold over lld is 1.7–12.6x, lower than the 2.4–16.1x on x86-64. Relative to the Threadripper, the M1 offers fewer but faster cores, a trade that favors a sequential linker. lld inherits much of the per-core speedup, linking the Firefox debug build in 3.26 s versus 4.43 s. mold, in contrast, loses more to the missing cores than it gains from the faster ones: the same link takes 1.02 s with 16 threads on the M1, up from 0.89 s with 32 threads on the Threadripper. Nevertheless, the results confirm that mold’s advantage holds across both machines.

7.3 Scalability

Figure 2 plots the link times of mold and lld for the Firefox debug build as a function of the worker thread count, from 1 to 64. mold speeds up steadily with thread count: 1.9x at 2 threads, 6.3x at 8, and 13.5x at 32, beyond which it plateaus. lld plateaus earlier, at 16 threads, with only 2.6x speedup. These results led us to set mold’s default cap at 32 threads and are consistent with lld’s default cap at 16 threads.

At one thread, the two linkers are not far apart (12.2 s for mold vs. 11.4 s for lld). The near-parity comes from two opposing effects that roughly cancel. mold’s parallel architecture has costs that a sequential linker does not pay, such as the eager full parsing of every archive member (§4.1) and the extra rounds of work some passes trade for parallelism (§4.7). Offsetting this, mold employs faster algorithms in some passes and utilizes compact data representations (§6.1). The balance tips decisively as threads are added, confirming that mold’s advantage comes from parallel scalability.

Table 6. End-to-end link times on ARM64 with 16 cores (seconds). Conventions are as in Table 4.

Program	mold	lld	gold	GNU ld	Speedup
Blender	0.24	0.59	3.37	6.17	2.5x
	1.41	3.39	38.47	46.29	2.4x
Chromium	1.04	4.70	—	—	4.5x
	2.43	9.97	—	—	4.1x
Clang	0.15	0.26	1.92	3.22	1.7x
	2.37	3.97	64.91	53.61	1.7x
ClickHouse	0.61	1.94	—	—	3.2x
	1.71	4.01	—	—	2.3x
Firefox	0.21	0.60	—	7.04	2.9x
	1.02	3.26	—	50.49	3.2x
Godot	0.11	0.24	1.55	3.04	2.2x
	0.59	1.00	14.56	15.88	1.7x
LibreOffice	0.25	0.55	2.01	13.83	2.2x
	0.66	1.86	17.69	—	2.8x
PyTorch	0.18	0.33	2.30	4.12	1.8x
	1.31	2.27	34.92	38.15	1.7x
TensorFlow	0.71	8.96	—	—	12.6x
	4.32	47.09	—	—	10.9x

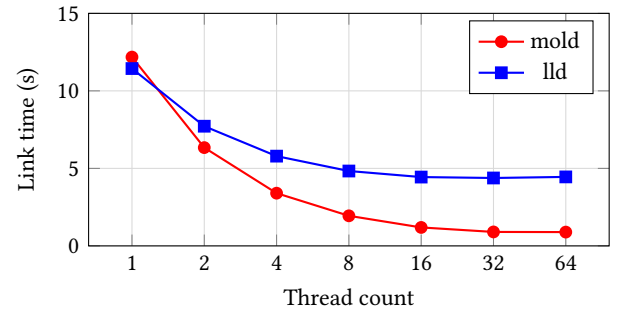
**Figure 2.** Scalability of mold and lld on the Firefox debug build.

Figure 3 visualizes the difference in CPU utilization over time: mold keeps many cores active for most of its 0.9 s execution, while lld runs predominantly on a single core with brief multi-core bursts, spread over 4.5 s. This is Amdahl’s law in action: lld’s serial portions dominate its execution time regardless of core count, while mold’s pervasive data parallelism keeps all worker threads busy.

Table 7 shows the CPU cost of parallel speedup. At 32 threads, mold achieves a 13.5x latency reduction by using 73% more CPU time than the one-thread run. At 64 threads, CPU time grows a further 80%, from 21.1 s to 38.2 s, with wall-clock time unchanged at 0.9 s. Adding threads beyond 32 increases CPU time without making the linker faster.

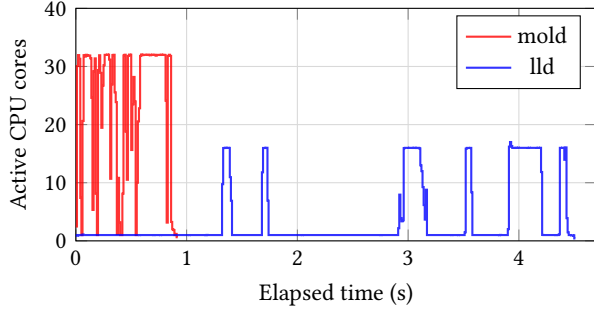


Figure 3. CPU core utilization over time for mold and lld on the Firefox debug build. Each step is the average number of active CPU cores over a 10 ms interval, computed from changes in the usage_usec counter of a dedicated cgroup.

Table 7. Cumulative CPU time (user + system) and wall-clock time on the Firefox debug build (seconds).

Threads	CPU time		Wall clock	
	mold	lld	mold	lld
1	12.2	11.4	12.2	11.4
8	14.3	13.4	1.9	4.8
16	16.0	14.7	1.2	4.4
32	21.1	18.1	0.9	4.4
64	38.2	25.5	0.9	4.5

Hardware performance counters, collected with Linux perf and AMD instruction-based sampling (IBS), attribute the plateau to two compounding effects. On the demand side, more workers traverse mold’s metadata at once, so the combined working set grows while cache capacity does not: from 32 to 64 threads, mold retires only 9% more instructions but incurs 37% more demand loads served from DRAM. On the supply side, the DRAM fill rate rises by only 38%, even though twice as many threads now wait on memory. For small random reads, DRAM throughput can saturate well before the data bus is fully utilized, and the 32-thread run already appears close to this limit: its sampled DRAM loads average 570 cycles (about 180 ns), roughly 40% higher than the machine’s unloaded random-access latency of about 130 ns. The additional requests at 64 threads therefore mostly increase waiting time rather than DRAM throughput: the mean latency inflates 1.9x, to about 1,070 cycles. The longer stalls reduce IPC from 0.85 to 0.53, explaining the extra CPU time. In sum, the larger working set causes 37% more demand loads to reach DRAM, so the 38% increase in DRAM fill rate is almost entirely absorbed by the additional traffic, leaving wall-clock time unchanged.

Table 8. Ablation study on the Firefox debug build. Each row shows the time spent in the named pass when it runs in the default parallel configuration (“Par.”) and when it is forced to run single-threaded while all other passes remain parallel (“Ser.”), along with the resulting end-to-end link time, all in seconds. “Overhead” is the increase in link time relative to fully parallel mold.

Serialized pass	Par.	Ser.	Total	Overhead
None (fully parallel)	—	—	0.91	—
Input file parsing	0.06	0.54	1.38	+52%
Symbol resolution	0.11	1.26	2.05	+125%
String merging	0.03	0.42	1.27	+40%
Section garbage collection	0.06	1.09	1.95	+114%
Relocation scanning	0.10	0.50	1.30	+43%
Output copy + reloc. apply	0.24	5.18	5.84	+542%
Build-ID computation	0.02	0.58	1.47	+62%

7.4 Ablation Study

To quantify the contribution of each parallelized pass, we measure mold’s performance when individual passes are forced to run single-threaded while the rest remain parallel. For each pass, Table 8 shows the time spent in the pass itself, run in parallel and serialized, and the effect on end-to-end link time, for the Firefox debug workload.

Every pass parallelizes, with pass-level speedups ranging from 5x to 29x at the default 32 threads. The end-to-end impact of serializing a pass is determined mostly by the pass’s share of total work. At one extreme, output copy adds +542% to the end-to-end link time when serialized. At the other, build-ID computation scales the best of all the passes (29x) yet costs only +62% when serialized, because it is a tiny fraction of the link. The cumulative effect of parallelizing *all* passes is what produces the large overall speedup: no single pass explains it, and even passes with modest individual overhead contribute to the end-to-end link time when serialized.

7.5 Where Does lld Spend Its Time?

To understand why lld takes 4.9x as long as mold, we profile both linkers on the Firefox debug workload using their built-in per-pass instrumentation. Table 9 shows the results. Since lld resolves symbols inside its file-parsing loop, its profile cannot report symbol resolution separately; the first row therefore combines parsing and symbol resolution.

The 3.53 s difference is distributed across the pipeline: no single category accounts for more than a third of it. Parsing and symbol resolution make the largest absolute contribution. lld spends 1.29 s, or 29% of its total time, in its largely sequential parse-and-resolve loop, versus 0.17 s for mold. This loop appears in Figure 3 as the single-core interval during lld’s first 1.3 s. Among passes with measurable times in

Table 9. Per-pass time breakdown comparison between mold and lld on the Firefox debug build (wall-clock seconds).

Pass	mold	lld	Speedup
Parsing + symbol resolution	0.17	1.29	7.6x
String merging	0.03	0.31	10.3x
Symbol version processing	0.00	0.24	—
Section garbage collection	0.06	0.85	14.2x
Relocation scanning	0.10	0.10	1.0x
Output copy + reloc. apply	0.23	0.50	2.2x
Other	0.31	1.14	3.7x
Total	0.90	4.43	4.9x

both linkers, section garbage collection has the largest relative gap, at 14.2x. lld’s mark phase is single-threaded and takes 0.85 s, whereas mold marks and sweeps in parallel and completes the pass in 0.06 s. Symbol version processing is also serial in lld and costs 0.24 s; mold performs it in parallel, and its time rounds to 0.00 s at the precision shown.

Passes that lld also parallelizes show a mixed picture. Relocation scanning takes 0.10 s in both linkers, showing that mold does not have a uniform constant-factor advantage. String merging shows the widest of these gaps, at 10.3x: lld parallelizes only part of the pass and, even at one thread, takes 1.7 s versus mold’s 0.4 s. The output-copy and relocation-application pass is 2.2x faster in mold. lld parallelizes across output sections but not within them, so each section is processed by a single thread regardless of how much work it requires. This coarse granularity follows from lld’s parallel-for implementation: a parallel-for invoked from within another parallel-for executes serially. This leaves the computationally intensive `.eh_frame_hdr` section to a single thread, creating a serial tail. mold’s parallel runtime supports nested parallel-for loops, allowing it to parallelize both across and within output sections.

The remaining 26% of lld’s time, 1.14 s, is spread across smaller, mostly sequential steps such as section layout, local symbol processing, and address-dependent content finalization; the corresponding remainder in mold is 0.31 s. Overall, lld takes 4.9x as long as mold not because of one dominant bottleneck, but because serial or incompletely parallel work appears throughout the pipeline.

7.6 Range Extension Thunks

Range extension thunks are created during section layout computation, making it difficult to isolate their overhead as a separate pass in a profiler. We instead disabled thunk creation in both mold and lld and compared link times with and without it. The resulting executables are broken, since out-of-range branches no longer reach their targets, but only the link time matters here. We measure the ARM64 Firefox debug link, cross-linked on the x86-64 machine.

Table 10. Range extension thunk overhead on the ARM64 Firefox debug build, measured by re-linking the same input with thunk creation disabled.

Linker	Normal	No thunks	Overhead
mold	1.05 s	0.98 s	+0.07 s
lld	5.56 s	4.55 s	+1.01 s

Table 10 shows the results. Thunks cost mold 0.07 s but lld 1.01 s, a 14x difference. To determine how much of the gap comes from the algorithm itself and how much from its parallelization, we repeated the measurement with both linkers restricted to one thread. Run single-threaded, mold’s thunk machinery costs 0.51 s versus lld’s 0.92 s, so mold’s linear scan algorithm with a cleanup pass beats lld’s repeated rescans by 1.8x on algorithm alone (§4.8). Parallelism accounts for the rest: mold scans each batch’s relocations in parallel, cutting its overhead 7x to 0.07 s, whereas lld’s scan is single-threaded, leaving its overhead essentially unchanged regardless of thread count.

8 Compatibility with GNU ld

As a compatibility test, we attempted to build all 19,422 packages in the Gentoo Linux package repository inside a container whose system linker was replaced with mold; 74 of them failed either to build or to pass their own test suite with mold but not with GNU ld.

Of the 74 failures, 25 involved bootloaders, firmware, kernels, and other non-userland software, which is currently outside mold’s scope (§9). Nine of the remaining 49 are false positives: the binaries work, but their test suites inspect the generated ELF files and reject harmless differences. The other 40 stem from a long tail of causes, including unsupported options such as `-b binary` and differences in library search behavior. Only two packages failed because mold’s parallel symbol resolution (§4.2) selects different archive members than GNU ld does.

9 Limitations

mold implements a subset of the GNU linker script language that is sufficient for almost all userland programs. It does not yet implement SECTIONS-based layout control or the MEMORY and OVERLAY commands, so mold cannot be used where explicit layout control is required, notably for Linux kernel development and many embedded systems.

mold currently supports only ELF, but little in its design is ELF-specific: every pass operates on sections, symbols, relocations, and archives, structures that other object formats such as COFF (Windows) and Mach-O (macOS) share. Indeed, an earlier version of mold included a Mach-O port built on the same parallel pass structure. We discontinued the port after Apple replaced its system linker with a new one

that is substantially faster [2], which reduced the demand for a third-party fast linker on macOS. We expect the techniques described in this paper to carry over to both COFF and Mach-O.

By default, mold limits itself to 32 threads, as we observe diminishing returns beyond this point (Figure 2). Investigating performance on high-core-count server processors, whose additional memory channels should raise the random-access ceiling observed in §7.3, is future work.

10 Related Work

The most closely related systems are the three major open-source ELF linkers, GNU ld, gold, and lld, described in §2; Table 1 compares which of their passes are parallelized. Song [30] analyzed lld’s performance characteristics and identified several phases that remain sequential, all of which mold parallelizes.

Wild [19] is a recent ELF linker written in Rust that also targets parallel linking performance. As of this writing (mid 2026), Wild is under active development and still lacks some features available in production linkers.

Levine’s *Linkers and Loaders* [21] is still the standard introduction to how linkers work, though parts of it no longer reflect current practice a quarter century after its publication.

Incremental linking takes a different route to fast turnaround. It patches the previous output in time proportional to the size of a change [26]; gold supports it, and Wild’s long-term goals include it. mold instead focuses on full-link speed. Incremental linking adds bookkeeping overhead to every link, and many common changes still force a full relink. For example, gold falls back to a full link when a function definition is removed or preallocated patch space runs out. A fast full link therefore remains important even for an incremental linker.

Split DWARF reduces linker work in another way. Instead of storing all debug information in object files, the compiler writes most of it to separate .dwo files, which the debugger reads directly and the linker never touches. The scheme was introduced in part to address long link times caused by large amounts of debugging information [6]. Adoption remains limited, as it is disabled by default and the .dwo files complicate build and debugging workflows. The approach is complementary to making the linker itself faster.

Another common workaround for long link times is to split a program into shared libraries, so that most changes relink one small library rather than the full executable. A documented example is Chromium’s component build [5]. The split comes at the cost of larger binaries, slower loading, and added build and code complexity. By making multi-gigabyte full links complete in a few seconds, mold makes such link-time-driven decomposition less compelling.

Tallam et al. [32] describe the ICF implementation in the gold linker. mold uses a different, parallel algorithm (§4.6).

ThinLTO [17] addresses the scalability of link-time optimization through partitioned compilation guided by module summaries. mold supports ThinLTO and full LTO as a black-box pass, orthogonal to its linking parallelism.

BOLT [25] is a post-link binary optimizer that reorganizes code layout based on profile data. BOLT operates on linker output and is complementary to mold.

Build systems such as Bazel [14], Buck [9], and Ninja [22] exploit parallelism at the level of compilation tasks. mold is complementary: it accelerates the linking step that typically cannot be parallelized at the build-system level because it is a single invocation.

11 Conclusion

We have presented mold, a production Unix/Linux linker that structures every major pass as a parallel-for loop over homogeneous data. On real-world workloads, mold is 2.4–16.1x faster than the state-of-the-art lld, even with mold’s portable system-level optimizations applied to lld, and up to 112x faster than the traditional GNU ld. This gap has no single cause. Prior linkers leave serial work in many different passes, and fully parallelizing these passes would require both new algorithms and substantial architectural changes. By building a new linker around pervasive parallelism, we show how every major linker pass can be parallelized and that doing so yields substantial end-to-end speedups.

Acknowledgments

We thank the anonymous reviewers for their feedback on this paper. We also thank the many contributors to mold, and the project’s GitHub sponsors, in particular Cybozu, G-Research, Mercury, Signal Slot, Ahrefs, Jinkyu Yi, Wei Wu, and Pedro Navarro, for their support. Claude Code was used to help draft and edit parts of this paper and to develop the benchmarking scripts for the evaluation.

References

- [1] Gene Myron Amdahl. 1967. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *Proceedings of the April 18-20, 1967, Spring Joint Computer Conference*. ACM, 483–485. doi:10.1145/1465482.1465560
- [2] Apple Inc. 2023. Platforms State of the Union. WWDC23 session 102. <https://developer.apple.com/videos/play/wwdc2023/102/>.
- [3] Arm Ltd. 2025. ELF for the Arm 64-bit Architecture (AArch64). <https://github.com/ARM-software/abi-aa/blob/main/aaelf64/aaelf64.rst>.
- [4] Guy E. Blelloch. 1990. *Prefix Sums and Their Applications*. Technical Report CMU-CS-90-190. School of Computer Science, Carnegie Mellon University.
- [5] Chromium Project. 2016. The Chrome Component Build. Chromium documentation. https://chromium.googlesource.com/chromium/src/+main/docs/component_build.md.
- [6] Cary Coutant. 2014. DWARF Package Files (Split DWARF, Part 5/5). DWARF Issue 140421.1, accepted for DWARF Version 5. <https://dwarfstd.org/issues/140421.1.html>.
- [7] DWARF Debugging Information Format Committee. 2017. DWARF Debugging Information Format, Version 5. <https://dwarfstd.org/dwa>

- rf5std.html.
- [8] Jason Evans. 2006. A Scalable Concurrent malloc(3) Implementation for FreeBSD. In *BSDCan*.
 - [9] Facebook, Inc. 2013. Buck: A Fast Build Tool. <https://buck.build/>.
 - [10] Philippe Flajolet, Éric Fusy, Olivier Gandouet, and Frédéric Meunier. 2007. HyperLogLog: The Analysis of a Near-Optimal Cardinality Estimation Algorithm. In *Proceedings of the Conference on Analysis of Algorithms (AofA)*. 127–146. doi:10.46298/dmtcs.3545
 - [11] Free Software Foundation. 2026. Index Files Speed Up GDB. *Debugging with GDB*. <https://sourceware.org/gdb/current/onlinedocs/gdb.html/Indexed-Files.html>.
 - [12] Sanjay Ghemawat and Paul Menage. 2007. TCMalloc: Thread-Caching Malloc. gperftools documentation. <https://gperftools.github.io/gperftools/tcmalloc.html>.
 - [13] GNU Binutils Project. 2025. GNU Binutils 2.44 Released. <https://lists.gnu.org/archive/html/info-gnu/2025-02/msg00001.html>.
 - [14] Google LLC. 2015. Bazel: A Fast, Scalable, Multi-Language Build System. <https://bazel.build/>.
 - [15] John Edward Hopcroft. 1971. An $n \log n$ Algorithm for Minimizing States in a Finite Automaton. In *Theory of Machines and Computations*. Academic Press, 189–196. doi:10.1016/B978-0-12-417750-5.50022-1
 - [16] Intel Corporation. 2021. oneAPI Threading Building Blocks (oneTBB). <https://github.com/oneapi-src/oneTBB>.
 - [17] Teresa Johnson, Mehdi Amini, and Xinliang David Li. 2017. ThinLTO: Scalable and Incremental LTO. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 111–121. doi:10.1109/CGO.2017.7863733
 - [18] Stephen Kell, Dominic P. Mulligan, and Peter Sewell. 2016. The Missing Link: Explaining ELF Static Linking, Semantically. In *Proceedings of the ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications (OOPSLA)*. 607–623. doi:10.1145/2983990.2983996
 - [19] David Lattimore. 2024. Wild: A Very Fast Linker Written in Rust. <https://github.com/wild-linker/wild>.
 - [20] Daan Leijen, Benjamin Zorn, and Leonardo de Moura. 2019. Mimalloc: Free List Sharding in Action. In *Proceedings of the Asian Symposium on Programming Languages and Systems (APLAS)*. 244–265. doi:10.1007/978-3-030-34175-6_13
 - [21] John R. Levine. 2000. *Linkers and Loaders*. Morgan Kaufmann. ISBN 978-1-55860-496-4.
 - [22] Evan Martin. 2012. Ninja: A Small Build System with a Focus on Speed. <https://ninja-build.org/>.
 - [23] Ralph Charles Merkle. 1988. A Digital Signature Based on a Conventional Encryption Function. In *Advances in Cryptology — CRYPTO '87*. Springer, 369–378. doi:10.1007/3-540-48184-2_32
 - [24] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. 2020. BLAKE3: One Function, Fast Everywhere. <https://github.com/BLAKE3-team/BLAKE3-specs>.
 - [25] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. 2019. BOLT: A Practical Binary Optimizer for Data Centers and Beyond. In *Proceedings of the IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. 2–14. doi:10.1109/CGO.2019.8661201
 - [26] Russell W. Quong and Mark A. Linton. 1991. Linking Programs Incrementally. *ACM Transactions on Programming Languages and Systems* 13, 1 (1991), 1–20. doi:10.1145/114005.102804
 - [27] Reproducible Builds Project. 2023. Reproducible Builds. <https://reproducible-builds.org/>.
 - [28] Alexander Schätzle, Antony Neu, Georg Lausen, and Martin Przyjacił-Zablocki. 2013. Large-Scale Bisimulation of RDF Graphs. In *Proceedings of the Fifth Workshop on Semantic Web Information Management (SWIM)*. ACM, 1–8. doi:10.1145/2484712.2484713
 - [29] Peter Smith. 2017. RFC: LLD Range Extension Thunks. llvm-dev mailing list. <https://lists.llvm.org/pipermail/llvm-dev/2017-January/108676.html>.
 - [30] Fangrui Song. 2021. Why isn't ld.lld faster? <https://maskray.me/blog/2021-12-19-why-isnt-ld.lld-faster>.
 - [31] Fangrui Song. 2026. Long Branches in Compilers, Assemblers, and Linkers. <https://maskray.me/blog/2026-01-25-long-branches-in-compilers-assemblers-and-linkers>.
 - [32] Sriraman Tallam, Cary Coutant, Ian Lance Taylor, Xinliang David Li, and Chris Demetriou. 2010. Safe ICF: Pointer Safe and Unwinding Aware Identical Code Folding in the Gold Linker. In *Proceedings of the GCC Developers' Summit*. <https://research.google/pubs/pub36912/>.
 - [33] Ian Lance Taylor. 2008. A New ELF Linker. In *Proceedings of the GCC Developers' Summit*. 129–136. <https://static.googleusercontent.com/media/research.google.com/en/intl/en/pubs/archive/34417.pdf>.
 - [34] TIS Committee. 1995. Tool Interface Standard (TIS) Executable and Linking Format (ELF) Specification, Version 1.2. <https://refspecs.linuxfoundation.org/elf/elf.pdf>.
 - [35] Rui Ueyama. 2017. lld: A Fast, Simple, and Portable Linker. 2017 LLVM Developers' Meeting. <https://llvm.org/devmtg/2017-10/slides/Ueyama-lld.pdf>.
 - [36] Boris Weisfeiler and Andrei Leman. 1968. The Reduction of a Graph to Canonical Form and the Algebra Which Appears Therein. *Nauchno-Tekhnicheskaya Informatsiya* 2, 9 (1968), 12–16. English translation: https://www.iti.zcu.cz/wl2018/pdf/wl_paper_translation.pdf.

A Artifact Appendix

A.1 Abstract

The artifact contains a container image with the evaluated linker configurations, captured inputs for the nine workloads in Table 3, and scripts reproducing Tables 4, 5, 7, 8, and 10 and Figures 2 and 3. It runs on x86-64 Linux and emits results in CSV format. Exact setup and execution commands are provided in the accompanying README.

A.2 Artifact check-list (meta-information)

- **Algorithm:** Data-parallel ELF linking
- **Program:** Nine workloads listed in Table 3; included with the artifact
- **Binary:** x86-64 Linux binaries for mold 2.42.0, lld 22.1.8, and GNU ld and gold 2.46.1; included in the container image
- **Data set:** Captured linker inputs for nine workloads
- **Run-time environment:** x86-64 Linux; Podman or Docker
- **Hardware:** At least 64 GiB RAM and 200 GiB free disk; 128 GiB RAM and NVMe storage recommended
- **Run-time state:** Otherwise-idle host; transparent huge pages is not disabled
- **Execution:** Shell and Python scripts; one warmup followed by five measured runs
- **Metrics:** Wall-clock and CPU time, peak RSS, and CPU utilization
- **Experiments:** Tables 3, 4, 5, 7, 8, and 10; Figures 2 and 3
- **How much time is needed to prepare workflow (approximately)?:** Network-dependent download and extraction; add about 30 minutes if rebuilding the container image
- **How much time is needed to complete experiments (approximately)?:** About 4 hours for the core results and 45 minutes for the supplemental results
- **Publicly available?:** Yes
- **Code licenses:** MIT, Apache-2.0 with LLVM exception, and GPL-3.0-or-later

- **Data licenses:** Upstream workload licenses; included with each workload
- **Archived (provide DOI)?:** [doi:10.5281/zenodo.21882261](https://doi.org/10.5281/zenodo.21882261)

A.3 Description

The artifact is archived on Zenodo at <https://doi.org/10.5281/zenodo.21882261>. Running it requires an x86-64 Linux machine with Podman or Docker, at least 64 GiB of RAM, and approximately 200 GiB of free space on an ext4 filesystem; 128 GiB of RAM and NVMe storage are recommended. All linker binaries and utilities are included in the container image, and the Containerfile records their provenance. Exact workload versions, licenses, and build configurations are documented in the README.

A.4 Installation

Unpack the artifact archive onto an ext4 filesystem and load the container image:

```
tar xf bench.tar.gz
cd bench
podman load -i mold-ae.tar
```

As a basic test, link the smallest workload once with every linker, from the unpacked benchmark directory (about a minute):

```
podman run --rm -v $PWD:/bench \
  --security-opt label=disable \
  -e PROJECTS=godot-release -e RUNS=1 \
  mold-ae with-thp ./tab4-bench.sh
```

It prints one CSV row per linker; a failed link would appear as an empty time field. With RUNS=1, the sole value is the warmup run and is intended only as a quick compatibility check.

A.5 Workflow and expected results

The README maps each paper result to the script that reproduces it and gives the exact container command. Timing experiments use one warmup run followed by five measured runs, whose median corresponds to the reported result. Absolute times depend on the machine, but mold should outperform lld on all workloads, continue scaling after lld plateaus, and slow down when any pass listed in Table 8 is serialized.

A.6 Notes

The benchmark inputs must reside on a bind-mounted ext4 filesystem. Storing them in the container overlaysfs disables huge-page-backed file mappings and introduces I/O overhead that would skew the comparison.